

《程序设计艺术与方法》课程实验报告

实验名称	实验一 STL 的熟悉与使用						
姓 名	李泽浩	学院	计算机与信息学院	班 级	软件工程 25-9 班	学 号	2025218895
实验日期			指导教师	曹力		成 绩	

一、实验目的和要求

(1) 掌握 C++中 STL 的容器类的使用。

(2) 掌握 C++中 STL 的算法类的使用。

二、实验预习内容

1. 预习 C++标准模板库（STL）中常见容器的基本概念及使用场景。

2.预习并熟悉 vector 和 list 的常用成员函数（如 push_back, insert, pop_back, clear 等）。

3.预习 STL 泛型算法的概念，了解 <algorithm> 头文件下的常用函数，如 find, sort, min_element, max_element 等的作用及迭代器传参方式。

三、实验项目摘要

1.练习 vector 和 list 的使用：定义空的 vector，生成 10 个随机数插入其中并遍历输出；在头部插入随机数并输出；使用泛型算法 find 查找随机数，找到则输出，否则插入尾部；使用 sort 排序并输出；删除尾部元素并输出；最后清空 vector。使用 list 容器重复上述实验步骤并观察异同。

2.练习泛型算法的使用：定义 vector 并插入 10 个随机数，使用 sort 分别按升序和降序排序并输出；练习使用 find 查找指定元素；使用 min_element 和 max_element 找出容器中的最小和最大元素并输出。

四、实验结果与分析（源程序及相关说明）

1. 练习 vector 和 list 的使用源程序

分析与注意点：

vector 在头部插入元素时需要使用 insert(v.begin(), val)，这会导致元素的移动，效率较低。

list 支持直接在头部插入，可以使用 push_front(val)，效率高。

标准库的 std::sort 泛型算法要求随机访问迭代器，vector 支持，但 list 是双向迭代器不支持。因此，对 list 排序必须调用其自身的成员函数 l.sort()。

```
#include <iostream>
#include <vector>
#include <list>
```

```
#include <algorithm>
#include <cstdlib>
#include <ctime>
using namespace std;
// 泛型函数，用于遍历输出容器元素
template <typename Container>
void printContainer(const Container& c) {
    for (auto i = c.begin(); i != c.end(); ++i) {
        cout << *i << " ";
    }
    cout << endl;
}

void testVector() {
    cout << "===== 1. Vector 实验 =====" << endl;
    vector<int> v;

    // 生成 10 个随机数插入到 vector 中
    for (int i = 0; i < 10; ++i) {
        v.push_back(rand() % 100);
    }
    cout << "初始 vector 元素: ";
    printContainer(v);
    // 在 vector 头部插入一个随机数
    int headNum = rand() % 100;
    v.insert(v.begin(), headNum);
    cout << "头部插入 " << headNum << " 后: ";
    printContainer(v);
    // 使用 find 查找某个随机数
    int targetNum = rand() % 100;
    auto itFind = find(v.begin(), v.end(), targetNum);
    if (itFind != v.end()) {
        cout << "查找 " << targetNum << " 成功, 元素存在!" << endl;
    } else {
        v.push_back(targetNum);
        cout << "查找 " << targetNum << " 失败, 已插入到尾部: ";
        printContainer(v);
    }
    // 使用泛型算法 sort 排序
    sort(v.begin(), v.end());
    cout << "sort 排序后: ";
    printContainer(v);
    // 删除尾部元素
    v.pop_back();
}
```

```
cout << "删除尾部元素后: ";
printContainer(v);
// 清空 vector
v.clear();
cout << "清空后 vector 大小: " << v.size() << endl;
}

void testList() {
    cout << "\n===== 2. List 实验 =====" << endl;
    list<int> l;

    // 生成 10 个随机数插入到 list 中
    for (int i = 0; i < 10; ++i) {
        l.push_back(rand() % 100);
    }

    cout << "初始 list 元素: ";
    printContainer(l);
    // 在 list 头部插入一个随机数
    int headNum = rand() % 100;
    l.push_front(headNum); // list 特有方法, 或者用 l.insert(l.begin(), headNum)
    cout << "头部插入 " << headNum << " 后: ";
    printContainer(l);
    // 使用 find 查找某个随机数
    int targetNum = rand() % 100;
    auto itFind = find(l.begin(), l.end(), targetNum);
    if (itFind != l.end()) {
        cout << "查找 " << targetNum << " 成功, 元素存在!" << endl;
    } else {
        l.push_back(targetNum);
        cout << "查找 " << targetNum << " 失败, 已插入到尾部: ";
        printContainer(l);
    }

    // list 的排序 (注意: list 不支持泛型 sort, 需使用成员函数)
    l.sort();
    cout << "list.sort() 排序后: ";
    printContainer(l);
    // 删除尾部元素
    l.pop_back();
    cout << "删除尾部元素后: ";
    printContainer(l);
    // 清空 list
    l.clear();
    cout << "清空后 list 大小: " << l.size() << endl;
}
```

2. 练习泛型算法的使用源程序

分析与注意点:

sort 默认是升序排序。若要实现降序排序,可以通过传入自定义的比较器如 `greater<int>()`,或者传递逆向迭代器 `rbegin()` 和 `rend()` 来实现。

获取最大最小值时,STL 提供的是 `min_element` 和 `max_element` 算法,它们返回的是指向最值的迭代器,所以输出结果时需要使用解引用符 `*` 获取具体的元素值。

```
void testGenericAlgorithms() {
    cout << "\n===== 3. 泛型算法综合实验 =====" << endl;
    vector<int> v;

    // 插入 10 个随机数
    for (int i = 0; i < 10; ++i) {
        v.push_back(rand() % 100);
    }

    cout << "初始序列: ";
    printContainer(v);

    // 升序排序
    sort(v.begin(), v.end());
    cout << "按升序排序后: ";
    printContainer(v);

    // 降序排序 (使用 greater<int> 或逆向迭代器)
    sort(v.begin(), v.end(), greater<int>());
    cout << "按降序排序后: ";
    printContainer(v);

    // 练习用 find 查找元素 (查找现有元素确保能找到)
    // 二分查找需要先排序,这里已经排序了,直接使用 find 查找一个存在的元素
    if (!v.empty()) {
        int existTarget = v[v.size() / 2];
        auto it = find(v.begin(), v.end(), existTarget);
        if (it != v.end()) {
            cout << "find 成功找到元素: " << *it << endl;
        }
    }

    // 用 min_element 和 max_element 找最大最小元素
    auto minIt = min_element(v.begin(), v.end());
    auto maxIt = max_element(v.begin(), v.end());

    if (minIt != v.end() && maxIt != v.end()) {
        cout << "容器中的最小元素为: " << *minIt << endl;
        cout << "容器中的最大元素为: " << *maxIt << endl;
    }
}
```

```
    }  
}  
  
int main() {  
    srand(static_cast<unsigned int>(time(0))); // 设置随机数种子  
    testVector();  
    testList();  
    testGenericAlgorithms();  
    return 0;  
}
```